

Helix AI: Agentic Architecture (In-Depth)

Seven-stage modernization workflow with guarded execution

Generated: 2026-04-05 01:24:58

This report documents how each agent/core module collaborates, what data is exchanged, and why this architecture is safe and extensible.

Project Snapshot	Value
Agent stages	Analyzer, Suggester, Critic, Planner, Execution, Validation, Report
Legacy rules tracked	15
Transform branches	15
Backend LOC	1158
Frontend LOC	2647
Test LOC	157
Detected endpoints	10
Migration transform rules	15
Rule benchmark pass	4/4
Unit test status	pass (10 tests)

How to read this document: each section starts with a simple explanation, then a deeper engineering explanation, then a practical checklist.

0. Agentic Pipeline at a Glance

Simple Explanation

In simple words: Analyzer finds issues, Suggester proposes fixes, Critic scores risk, Planner orders actions, Execution applies, Validation checks, Report explains.

Deep Explanation

The pipeline is sequential by design to keep state transitions easy to reason about and debug.

Each stage emits structured data that the next stage can consume without hidden side effects.

The architecture aligns with your original diagram while tightening contracts between stages.

Practical Checklist

- Explain each stage in one sentence.
- Emphasize explicit hand-off data.
- Mention that report and diff close the loop.

Code References

```
agents/analyzer.py
agents/suggester.py
agents/critic.py
agents/planner.py
core/execution.py
core/validation.py
core/report_generator.py
```

1. Agent Responsibilities (Detailed)

Simple Explanation

In simple words: each agent has one job, so errors are easier to isolate.

Deep Explanation

Analyzer: pattern detection, source-version hinting, risk mode assignment, and lightweight complexity estimate.

Suggester: deterministic mapping from issue id to actionable transform candidate.

Critic: safety score and warning state based on transform type and confidence.

Planner: final execution ordering based on priority and safety score.

Practical Checklist

- Keep single-responsibility boundaries.
- Avoid mixing transformation logic into analysis stage.
- Avoid direct code edits outside execution core.

Code References

```
agents/analyzer.py
agents/suggester.py
agents/critic.py
agents/planner.py
```

2. Data Contracts Between Agents

Simple Explanation

In simple words: every stage returns JSON-like structured objects.

Deep Explanation

Analyzer output includes success flag, probable source version, issue list, risk score, mode, and semantic risk set.

Suggester output is a list of normalized suggestion records with id, type, confidence, priority, line, and reasoning.

Critic output is a list of per-suggestion statuses and safety scores.

Planner output includes selected_plans and candidates for traceability.

Practical Checklist

- Keep field names stable across versions.
- Default missing lists to empty lists, not null.
- Log schema mismatches quickly during integration.

Code References

```
main.py  
agents/*.py
```

3. Orchestration and Control Flow

Simple Explanation

In simple words: API endpoint `/refactor` is the orchestrator that runs the full chain end-to-end.

Deep Explanation

The orchestrator first analyzes, then plans, then executes, then validates, then stores run data and returns result payload.

If validation fails, rollback to original code is enforced before response.

If ML is available, it runs as auxiliary output and does not override deterministic final code.

Practical Checklist

- Preserve rollback on validation failure.
- Keep ML branch side-effect free on deterministic code path.
- Always include logs and diff in response.

Code References

`main.py`

4. Safety Layers

Simple Explanation

In simple words: safety is not a single check; it is multiple guardrails in sequence.

Deep Explanation

Layer 1: analyzer risk modes include SAFE, RESTRICTED, BLOCKED.

Layer 2: critic reduces scores for semantic and low-confidence changes.

Layer 3: validation reparses Python and checks leftover legacy patterns.

Layer 4: behavior checks ensure key function names are not accidentally removed.

Practical Checklist

- Design for fail-closed behavior.
- Require explicit user review for warning-heavy runs.
- Store warnings in run records.

Code References

`agents/analyzer.py`
`agents/critic.py`
`core/validation.py`

5. ML Placement in Agentic Architecture

Simple Explanation

In simple words: ML is a helper brain, not the only brain.

Deep Explanation

Reasoner availability is environment controlled and visible to user through `/model-status`.

ML inference uses adapter path + base model and can produce a side-channel modernization draft.

Final production-safe output remains the validated deterministic output unless you intentionally redesign the authority model.

Practical Checklist

- Use ML for ranking and fallback suggestions first.
- Avoid giving ML direct write authority over accepted output.
- Gate ML outputs through same validator in future extensions.

Code References

```
core/ml_reasoner.py  
main.py
```

6. State, Persistence, and Observability

Simple Explanation

In simple words: each run is saved with enough data to replay and audit what happened.

Deep Explanation

Run store currently has 59 persisted records in the local history file (count can grow during usage).

Saved fields include original/new code, analysis, validation, applied transforms, diff, and report text.

This enables a simple but effective audit trail for demo, viva, and debugging.

Practical Checklist

- Keep run payload complete enough for debugging.
- Limit run list endpoint output to summary fields for UI performance.
- Provide run detail endpoint for deep inspection.

Code References

```
core/run_store.py  
main.py
```

7. Scale and Future Agent Evolution

Simple Explanation

In simple words: if usage grows, separate agent steps into workers while preserving current contracts.

Deep Explanation

Current in-process orchestration is ideal for prototype speed and low operational overhead.

Future architecture can split into queue-based workers for analyzer/planner/execution with the same message schema.

Versioned schemas should be introduced before distributed execution to avoid drift.

Practical Checklist

- Stabilize schema before scaling architecture.
- Add structured event logs for each stage.
- Keep deterministic stage idempotent for retries.

Code References

```
docs/ARCHITECTURE.md
```

Viva Quick Answers

Q: Why split into multiple agents?

Separation of concerns keeps behavior explainable and makes debugging easier.

Q: Where is orchestration done?

In main.py endpoints, especially /analyze and /refactor.

Q: What prevents unsafe changes?

Risk mode gating + critic warnings + validation gate + rollback.

Q: How does ML fit this architecture?

ML provides auxiliary modernization output and status, while deterministic validated output remains authoritative.

Q: How can this architecture scale?

By preserving contracts and moving stages into queue workers when needed.

From Scratch Learning Track

This chapter is written for beginners first. Each module starts with very simple meaning, then engineering depth, then practical action.

Module 1: Agent Role Isolation

Simple view: agent role isolation explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, agent role isolation connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 2: Contract-First Stage Outputs

Simple view: contract-first stage outputs explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, contract-first stage outputs connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 3: Risk-Aware Orchestration Order

Simple view: risk-aware orchestration order explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, risk-aware orchestration order connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 4: Planner And Critic Interaction

Simple view: planner and critic interaction explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, planner and critic interaction connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 5: Execution Authority And Validation Authority

Simple view: execution authority and validation authority explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, execution authority and validation authority connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 6: Status Visibility And Observability

Simple view: status visibility and observability explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, status visibility and observability connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 7: Run History And Traceability

Simple view: run history and traceability explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, run history and traceability connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 8: Model-Assisted Side Channel Design

Simple view: model-assisted side channel design explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, model-assisted side channel design connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 9: Blocked Mode And Restricted Mode Handling

Simple view: blocked mode and restricted mode handling explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, blocked mode and restricted mode handling connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 10: Schema Evolution Strategy

Simple view: schema evolution strategy explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, schema evolution strategy connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 11: Future Queue-Based Scaling Path

Simple view: future queue-based scaling path explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, future queue-based scaling path connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 12: Agentic Governance And Safety Policy

Simple view: agentic governance and safety policy explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, agentic governance and safety policy connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and

evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Hands-On Practical Labs

1. Lab 1: Baseline Input Audit

Start from a legacy snippet and identify exactly what will break on modern Python.

- Collect one legacy input sample.
- Run /analyze and record issue ids.
- Classify each issue as syntax or semantic risk.
- Write one paragraph on expected modernization output.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

2. Lab 2: Analyze Output Interpretation

Understand how analysis payload maps to later planner and execution actions.

- Inspect probable_source_version and mode.
- Read legacy_issues list line by line.
- Map each issue id to a suggestion id.
- Explain why some items should become warning.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

3. Lab 3: Critic + Planner Trace

Trace how safety score and priority become selected_plans.

- Run analyze output through critique logic.
- Compute final score manually for two suggestions.
- Verify execution order in planner output.
- Compare manual and program outputs.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

4. Lab 4: Execution and Diff Review

Run modernization and inspect unified diff before applying changes.

- Execute /refactor on sample input.
- Inspect added and removed lines.
- Confirm each line links to a planned transform.
- Document one risky change for manual review.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

5. Lab 5: Validation Failure Drill

Learn fail-closed behavior by forcing a leftover-legacy construct.

- Create code where one legacy construct remains.
- Run validation and capture failure stage.
- Explain rollback behavior and user impact.
- Fix rule and rerun to green state.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

6. Lab 6: Run History and Reproducibility

Use run history to prove the pipeline is auditable and repeatable.

- Generate at least two runs with different inputs.
- Use /runs and /runs/{id} endpoints.
- Re-open one run in UI and inspect report tab.
- Write traceability summary notes.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

7. Lab 7: Topic-Specific Deep Task

Perform a deeper exercise related to agentic concerns and document engineering tradeoffs.

- Choose one module strongly tied to this topic.
- Read code and summarize control flow.
- List strengths, risks, and missing checks.
- Propose one measurable improvement.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

8. Lab 8: Evaluation and Evidence Pack

Prepare evidence artifacts you can use in submission and viva.

- Run tests and benchmark script.
- Capture summary metrics and limitations.
- Align report claims with actual metrics.
- Store outputs in a clean docs folder.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

9. Lab 9: Explain to Beginner

Re-explain one complex part in simple words without losing correctness.

- Pick one algorithmic decision.
- Write a beginner explanation in 8 lines.
- Write an expert explanation in 8 lines.
- Ensure both versions match same truth.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

10. Lab 10: Full Demo Rehearsal

Do complete end-to-end dry run as if in final presentation.

- Start server and open UI.
- Run one safe case and one warning case.
- Show diff, report, and run history.
- Conclude with roadmap and limitations.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

Troubleshooting Playbook

Issue	Likely Cause	How To Fix
Analysis returns no issues unexpectedly	Input pattern not covered by rules	Add/adjust analyzer rule and unit test.
Planner selects zero actions	Suggestions became NOOP or filtered by status	Inspect suggestion ids, critique map, and planner filters.
Validation fails after execution	Legacy leftovers or syntax problems remain	Inspect failure stage and patch transform logic.
Diff looks confusing to users	Output formatting or context lines are unclear	Improve diff rendering and report hints.
Run history missing expected data	RunStore payload fields not persisted	Verify run payload contract in save_run call.
ML shows unavailable	Adapter path missing or model disabled	Check env flags and adapter directory path.
UI button state feels wrong	State machine and action label out of sync	Trace state.primaryAction transitions.
Performance slows on large input	Heavy rendering or repeated processing	Paginate rendering and profile hot paths.
Benchmarks look too optimistic	Benchmark set too small or narrow	Expand dataset and stratify by pattern type.
Viva answers feel weak	No concrete evidence references	Quote tests, benchmark numbers, and code paths.
Topic-specific confusion in stage contracts	Corresponding logic in concrete file and flow	Add diagram plus file-level walkthrough.
Claims sound exaggerated	Scope and limitations not stated	Add explicit limitations and roadmap stages.

Glossary (Simple Words, Technical Meaning)

1. Legacy Pattern

An old syntax or API usage from earlier Python versions.

2. Modernization

Converting legacy code to current Python-compatible code.

3. Deterministic

Same input always gives same output.

4. Semantic Risk

Code may still run but behavior could change.

5. Validation Gate

Final checks that decide pass or rollback.

6. Diff-First Review

Inspect exact changes before applying to editor.

7. Analyzer

Stage that detects issues and risk indicators.

8. Suggester

Stage that maps detected issues to upgrades.

9. Critic

Stage that assigns safety scores and warnings.

10. Planner

Stage that chooses execution order.

11. Execution Core

Stage that applies selected transforms.

12. RunStore

Persistence layer for run history records.

13. LOCA (LOC)

Lines of code metric for rough project size.

14. Benchmark

Set of test cases used to measure pass rate.

15. Smoke Test

Small test set for quick health verification.

16. Rollback

Revert to original code when validation fails.

17. Adapter

LoRA fine-tuned parameter delta for base model.

18. Base Model

Original pretrained model used before fine-tuning.

19. Confidence

Estimated reliability signal for a suggestion.

20. Traceability

Ability to explain what changed and why.

21. Agent Contract

Structured input/output schema between stages.

22. Orchestration

Coordinating stage execution in correct order.

23. Mode SAFE/RESTRICTED/BLOCKED

Risk-level operating states from analyzer.

24. Candidate Plan

Potential transform item before final selection.

25. Authority Model

Who decides final output acceptance.

Extended Viva and Interview Q&A

Q1: What is the core goal of this report?

To explain this topic from beginner level to implementation depth with real project evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q2: How should I present this in viva?

Start with simple explanation, then show one concrete code path, then show one metric and one limitation.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q3: How do I avoid vague answers?

Always tie statements to file paths, endpoints, tests, or measured outputs.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q4: What if evaluator asks about limitations?

Answer directly and show roadmap with near-term measurable upgrades.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q5: How do I show project credibility?

Use deterministic workflow, validation behavior, and test/benchmark evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q6: Why use simple language in technical docs?

It improves clarity and comprehension without reducing technical correctness.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q7: How should I practice this topic?

Run one lab task, inspect code, and explain output in your own words.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q8: How does this topic connect to full system?

Each topic is one layer of the same end-to-end modernization pipeline.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q9: Why separate analyzer and execution?

Detection and rewriting have different responsibilities and failure modes.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q10: How does the planner add value?

It imposes deterministic order and allows risk-aware selection.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q11: Can agents be distributed later?

Yes, contracts are already explicit enough for queue-based scaling.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Code Walkthrough Appendix (With Explanation)

This appendix connects explanation to real files. Each excerpt includes simple-language meaning, technical interpretation, and why it matters before showing code lines.

Excerpt 1: Analyzer Stage

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/analyzer.py

Simple explanation: this block handles legacy pattern detection and risk mode assignment. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=0, classes=0, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Regex operations are present, so edge-case pattern handling should be tested carefully.
- Risk/warning language exists in this block, so it affects safety signaling.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import ast
0002: import re
0003: from typing import Any, Dict, List
0004:
0005: from agents.base_agent import BaseAgent
0006:
0007:
0008: LEGACY_RULES = [
0009:     {
0010:         "id": "backtick_repr",
0011:         "pattern": re.compile(r"`[^\\n]+`"),
0012:         "source_hint": "Python 1.x/2.x",
0013:         "message": "Backtick repr syntax detected.",
0014:         "replacement": "Replace `expr` with repr(expr).",
0015:         "risk": "MEDIUM",
0016:     },
0017:     {
0018:         "id": "apply_builtin",
0019:         "pattern": re.compile(r"\\bapply\\s*\\("),
0020:         "source_hint": "Python 1.x/2.x",
0021:         "message": "Legacy apply(...) builtin detected.",
0022:         "replacement": "Rewrite apply(fn, args[, kwargs]) as fn(*args[, **kwargs]).",
0023:         "risk": "MEDIUM",
0024:     },
0025:     {
0026:         "id": "print_statement",
0027:         "pattern": re.compile(r"\\s*print\\s+[^(\\n].*$", re.MULTILINE),
0028:         "source_hint": "Python 2.x",
0029:         "message": "Legacy print statement detected.",
0030:         "replacement": "Wrap values with print(...).",
0031:         "risk": "LOW",
0032:     },
0033:     {
0034:         "id": "xrange_usage",
0035:         "pattern": re.compile(r"\\bxrange\\s*\\("),
```

```

0036:         "source_hint": "Python 2.x",
0037:         "message": "xrange is not available in modern Python.",
0038:         "replacement": "Replace xrange(...) with range(...).",
0039:         "risk": "LOW",
0040:     },
0041:     {
0042:         "id": "raw_input_usage",
0043:         "pattern": re.compile(r"\braw_input\s*\("),
0044:         "source_hint": "Python 2.x",
0045:         "message": "raw_input was renamed in Python 3.",
0046:         "replacement": "Replace raw_input(...) with input(...).",
0047:         "risk": "LOW",
0048:     },
0049:     {
0050:         "id": "except_comma",
0051:         "pattern": re.compile(r"\s*except\s+([^\n]+),\s*[A-Za-z_]\w*\s*:", re.MULTILINE),
0052:         "source_hint": "Python 2.x",
0053:         "message": "Old exception binding syntax detected.",
0054:         "replacement": "Convert except X, e: to except X as e:.",
0055:         "risk": "LOW",
0056:     },
0057:     {
0058:         "id": "not_equal_operator",
0059:         "pattern": re.compile(r"<>"),
0060:         "source_hint": "Python 2.x",
0061:         "message": "Legacy not-equal operator detected.",
0062:         "replacement": "Replace <> with !=",
0063:         "risk": "LOW",
0064:     },
0065:     {
0066:         "id": "iteritems_usage",
0067:         "pattern": re.compile(r"\.iteritems\s*\("),
0068:         "source_hint": "Python 2.x",
0069:         "message": "iteritems returns a Python 2 iterator API.",
0070:         "replacement": "Replace iteritems() with items().",
0071:         "risk": "LOW",
0072:     },
0073:     {
0074:         "id": "iterkeys_usage",
0075:         "pattern": re.compile(r"\.iterkeys\s*\("),
0076:         "source_hint": "Python 2.x",
0077:         "message": "iterkeys returns a Python 2 iterator API.",
0078:         "replacement": "Replace iterkeys() with keys().",
0079:         "risk": "LOW",
0080:     },
0081:     {
0082:         "id": "itervalues_usage",
0083:         "pattern": re.compile(r"\.itervalues\s*\("),
0084:         "source_hint": "Python 2.x",
0085:         "message": "itervalues returns a Python 2 iterator API.",
0086:         "replacement": "Replace itervalues() with values().",
0087:         "risk": "LOW",
0088:     },
0089:     {
0090:         "id": "has_key_usage",
0091:         "pattern": re.compile(r"\.has_key\s*\("),
0092:         "source_hint": "Python 2.x",
0093:         "message": "dict.has_key was removed.",
0094:         "replacement": "Rewrite d.has_key(x) as x in d.",
0095:         "risk": "MEDIUM",
0096:     },
0097:     {
0098:         "id": "unicode_type",
0099:         "pattern": re.compile(r"\bunicode\b"),
0100:         "source_hint": "Python 2.x",
0101:         "message": "unicode type detected.",
0102:         "replacement": "Replace unicode with str where compatible.",
0103:         "risk": "MEDIUM",
0104:     },

```

```

0105:     {
0106:         "id": "basestring_type",
0107:         "pattern": re.compile(r"\bbasestring\b"),
0108:         "source_hint": "Python 2.x",
0109:         "message": "basestring type detected.",
0110:         "replacement": "Replace basestring with str.",
0111:         "risk": "MEDIUM",
0112:     },
0113:     {
0114:         "id": "long_type",
0115:         "pattern": re.compile(r"\blong\b"),
0116:         "source_hint": "Python 2.x",
0117:         "message": "long type detected.",
0118:         "replacement": "Replace long with int.",
0119:         "risk": "LOW",
0120:     },
0121:     {
0122:         "id": "exec_statement",
0123:         "pattern": re.compile(r"^s*exec\s+[(\n].*$", re.MULTILINE),
0124:         "source_hint": "Python 1.x/2.x",
0125:         "message": "Legacy exec statement detected.",
0126:         "replacement": "Convert exec statement syntax to exec(...).",
0127:         "risk": "HIGH",
0128:     },
0129: ]
0130:

```

Excerpt 2: Suggester Stage

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/suggester.py

Simple explanation: this block handles issue-to-transform mapping and confidence metadata. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=0, classes=0, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: from typing import Any, Dict, List
0002:
0003: from agents.base_agent import BaseAgent
0004:
0005:
0006: ISSUE_TO_SUGGESTION = {
0007:     "backtick_repr": {
0008:         "id": "upgrade_backtick_repr",
0009:         "type": "SEMANTIC_UPGRADE",
0010:         "target": "backtick repr",
0011:         "suggestion": "repr_call",
0012:         "reasoning": "Backticks were an old repr shorthand and should become repr(...).",
0013:         "confidence": 0.9,
0014:         "priority": 12,
0015:     },
0016:     "apply_builtin": {
0017:         "id": "upgrade_apply_builtin",
0018:         "type": "SEMANTIC_UPGRADE",
0019:         "target": "apply builtin",
0020:         "suggestion": "star_call",
0021:         "reasoning": "apply(fn, args[, kwargs]) is now expressed as fn(*args[, **kwargs]).",
0022:         "confidence": 0.9,
0023:         "priority": 18,
0024:     },
0025:     "print_statement": {
0026:         "id": "upgrade_print_statement",
0027:         "type": "SYNTAX_UPGRADE",
0028:         "target": "print statement",
0029:         "suggestion": "print_function",
0030:         "reasoning": "Convert legacy print statements to Python 3 print(...) calls.",
0031:         "confidence": 0.98,
0032:         "priority": 10,
0033:     },
0034:     "xrange_usage": {
0035:         "id": "upgrade_xrange",
0036:         "type": "API_UPGRADE",
0037:         "target": "xrange",
0038:         "suggestion": "range",
0039:         "reasoning": "xrange was removed; range provides the modern equivalent.",
0040:         "confidence": 0.99,
0041:         "priority": 20,
0042:     },
0043:     "raw_input_usage": {
0044:         "id": "upgrade_raw_input",
```

```

0045:         "type": "API_UPGRADE",
0046:         "target": "raw_input",
0047:         "suggestion": "input",
0048:         "reasoning": "raw_input was renamed to input in Python 3.",
0049:         "confidence": 0.99,
0050:         "priority": 20,
0051:     },
0052:     "except_comma": {
0053:         "id": "upgrade_except_syntax",
0054:         "type": "SYNTAX_UPGRADE",
0055:         "target": "except syntax",
0056:         "suggestion": "except as",
0057:         "reasoning": "Modern Python binds exceptions with `as`.",
0058:         "confidence": 0.97,
0059:         "priority": 15,
0060:     },
0061:     "not_equal_operator": {
0062:         "id": "upgrade_not_equal",
0063:         "type": "SYNTAX_UPGRADE",
0064:         "target": "<>",
0065:         "suggestion": "!=",
0066:         "reasoning": "The <> operator is obsolete.",
0067:         "confidence": 1.0,
0068:         "priority": 10,
0069:     },
0070:     "iteritems_usage": {
0071:         "id": "upgrade_iteritems",
0072:         "type": "API_UPGRADE",
0073:         "target": "iteritems",
0074:         "suggestion": "items",
0075:         "reasoning": "Use items() for modern dictionary iteration.",
0076:         "confidence": 0.98,
0077:         "priority": 25,
0078:     },
0079:     "iterkeys_usage": {
0080:         "id": "upgrade_iterkeys",
0081:         "type": "API_UPGRADE",
0082:         "target": "iterkeys",
0083:         "suggestion": "keys",
0084:         "reasoning": "Use keys() for modern dictionary iteration.",
0085:         "confidence": 0.98,
0086:         "priority": 25,
0087:     },
0088:     "itervalues_usage": {
0089:         "id": "upgrade_itervalues",
0090:         "type": "API_UPGRADE",
0091:         "target": "itervalues",
0092:         "suggestion": "values",
0093:         "reasoning": "Use values() for modern dictionary iteration.",
0094:         "confidence": 0.98,
0095:         "priority": 25,
0096:     },
0097:     "has_key_usage": {
0098:         "id": "upgrade_has_key",
0099:         "type": "SEMANTIC_UPGRADE",
0100:         "target": "has_key",
0101:         "suggestion": "in_operator",
0102:         "reasoning": "Replace deprecated has_key checks with membership tests.",
0103:         "confidence": 0.92,
0104:         "priority": 30,
0105:     },
0106:     "unicode_type": {
0107:         "id": "upgrade_unicode",
0108:         "type": "TYPE_UPGRADE",
0109:         "target": "unicode",
0110:         "suggestion": "str",
0111:         "reasoning": "Python 3 uses str for text values.",
0112:         "confidence": 0.88,
0113:         "priority": 35,

```

```

0114: },
0115: "basestring_type": {
0116:     "id": "upgrade_basestring",
0117:     "type": "TYPE_UPGRADE",
0118:     "target": "basestring",
0119:     "suggestion": "str",
0120:     "reasoning": "basestring no longer exists in Python 3.",
0121:     "confidence": 0.9,
0122:     "priority": 35,
0123: },
0124: "long_type": {
0125:     "id": "upgrade_long",
0126:     "type": "TYPE_UPGRADE",
0127:     "target": "long",
0128:     "suggestion": "int",
0129:     "reasoning": "int subsumes long in modern Python.",
0130:     "confidence": 0.96,

```

Excerpt 3: Planner + Critic Stage

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/planner.py

Simple explanation: this block handles execution ordering and transform selection. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=3, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: `__init__`, `process`, `process`.
- Risk/warning language exists in this block, so it affects safety signaling.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: from typing import Any, Dict, List
0002:
0003: from agents.base_agent import BaseAgent
0004:
0005:
0006: class PlannerAgent(BaseAgent):
0007:     def __init__(self):
0008:         super().__init__("Planner")
0009:
0010:     async def process(self, suggestions: List[Dict[str, Any]], critiques: Dict[str, Any]) -> Dict:
0011:         critique_map = {item["suggestion_id"]: item for item in critiques["critiques"]}
0012:         candidates = []
0013:
0014:         for suggestion in suggestions:
0015:             critique = critique_map.get(suggestion["id"])
0016:             if not critique:
0017:                 continue
0018:             if critique["status"] not in {"APPROVED", "WARNING"}:
0019:                 continue
0020:
0021:             final_score = critique["safety_score"] + max(0, 50 - suggestion.get("priority", 50))
0022:             candidates.append(
0023:                 {
0024:                     "suggestion": suggestion,
0025:                     "critique": critique,
0026:                     "safety_score": final_score,
0027:                     "execution_order": suggestion.get("priority", 50),
0028:                 }
0029:             )
0030:
0031:             candidates.sort(key=lambda item: (item["execution_order"], -item["safety_score"], item["suggestion_id"]))
0032:             selected_plans = [item for item in candidates if item["suggestion"]["type"] != "NOOP"]
0033:
0034:         return {
0035:             "selected_plan": selected_plans[0] if selected_plans else None,
0036:             "selected_plans": selected_plans,
0037:             "total_candidates": len(suggestions),
0038:             "approved_count": len(selected_plans),
0039:             "candidates": candidates,
0040:         }
```

Excerpt 4: Orchestration

File: /Users/rutwikbhondave/Desktop/Helix AI Project/main.py

Simple explanation: this block handles API orchestration and full modernization flow control. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=4, classes=0, endpoint decorators=2.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: analyze_code, refactor_code, analyze_code, refactor_code.
- Endpoint decorators in excerpt: app.post, app.post.
- Risk/warning language exists in this block, so it affects safety signaling.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0180:             logging.error("Failed to read inquiries: %s", exc)
0181:             inquiries = []
0182:
0183:             inquiries.append(inquiry_data)
0184:
0185:             with open(file_path, "w", encoding="utf-8") as file:
0186:                 json.dump(inquiries, file, indent=4)
0187:
0188:             return JsonResponse(content={"success": True, "message": "Migration inquiry received."})
0189:     except Exception as exc:
0190:         logging.error("Form submission failed: %s", exc)
0191:         raise HTTPException(status_code=500, detail="Transmission failed")
0192:
0193:
0194: @app.post("/analyze")
0195: async def analyze_code(request: CodeRequest):
0196:     try:
0197:         analysis = await analyzer.process(request.code)
0198:         suggestions = await suggester.process(analysis, request.code)
0199:         critiques = await critic.process(suggestions)
0200:         plan_result = await planner.process(suggestions, critiques)
0201:         ml_status = reasoner.explain_status()
0202:
0203:         return JsonResponse(
0204:             content={
0205:                 "analysis": analysis,
0206:                 "suggestions": suggestions,
0207:                 "critiques": critiques,
0208:                 "plan": plan_result,
0209:                 "ml_reasoner": ml_status,
0210:             }
0211:         )
0212:     except Exception as exc:
0213:         logging.error("Analysis failed: %s", exc)
0214:         raise HTTPException(status_code=500, detail=str(exc))
0215:
0216:
0217: @app.post("/refactor")
0218: async def refactor_code(request: CodeRequest):
```

```

0219:     try:
0220:         logs = ["--- Starting Legacy Python Modernization Pipeline ---"]
0221:         run_id = None
0222:
0223:         analysis = await analyzer.process(request.code)
0224:         if not analysis.get("success"):
0225:             return JSONResponse(content={"error": "Analysis failed", "details": analysis.get("error")})
0226:
0227:         logs.append(f"Analyzer: probable source version = {analysis.get('probable_source_version')}")
0228:         logs.append(f"Analyzer: detected {len(analysis.get('legacy_issues', []))} legacy issues.")
0229:
0230:         if analysis.get("mode") == "BLOCKED":
0231:             logs.append("Execution blocked due to critical dynamic execution risk.")
0232:             return JSONResponse(content={"error": "Blocked", "logs": logs, "analysis": analysis})
0233:
0234:         suggestions = await suggester.process(analysis, request.code)
0235:         critique = await critic.process(suggestions)
0236:         plan_result = await planner.process(suggestions, critique)
0237:         selected_plans = plan_result.get("selected_plans", [])
0238:         ml_result = None
0239:
0240:         logs.append(f"Suggester: generated {len(suggestions)} migration suggestions.")
0241:         logs.append(f"Planner: selected {len(selected_plans)} transformations for execution.")
0242:
0243:         new_code = request.code
0244:         validation_result = {"success": False, "stage": "SKIPPED", "message": "No changes applied"}
0245:         validation_success = False
0246:
0247:         if selected_plans:
0248:             new_code = executor.apply_changes(request.code, selected_plans)
0249:             logs.append("Execution: transformations applied sequentially.")
0250:
0251:             validation_result = validator.validate(request.code, new_code)
0252:             if validation_result["success"]:
0253:                 validation_success = True
0254:                 logs.append("Validation: modernization passed all configured checks.")
0255:             else:
0256:                 logs.append(f"Validation failed: {validation_result.get('error')}")
0257:                 logs.append("Rollback: reverting to original code.")
0258:                 new_code = request.code
0259:         else:
0260:             logs.append("Planner: no executable transformations were selected.")
0261:
0262:         if reasoner.is_available():
0263:             ml_result = reasoner.modernize(request.code)
0264:             if ml_result.available and ml_result.output:
0265:                 logs.append("ML Reasoner: generated auxiliary modernization output.")
0266:             elif ml_result.error:
0267:                 logs.append(f"ML Reasoner unavailable: {ml_result.error}")
0268:
0269:         diff = "\n".join(
0270:             difflib.unified_diff(
0271:                 request.code.splitlines(),
0272:                 new_code.splitlines(),
0273:                 fromfile="legacy.py",
0274:                 tofile="modernized.py",
0275:                 lineterm="",
0276:             )
0277:         )
0278:
0279:         report = reporter.generate_report(
0280:             analysis=analysis,
0281:             selected_plans=selected_plans,
0282:             success=validation_success,
0283:             validation_logs=logs,
0284:             validation_result=validation_result,
0285:         )
0286:         run_record = run_store.save_run(
0287:             {

```

```
0288:         "probable_source_version": analysis.get("probable_source_version"),
0289:         "risk_score": analysis.get("risk_score"),
0290:         "legacy_issue_count": len(analysis.get("legacy_issues", [])),
```